Capstone Project

End of Module Reflective Portfolio

Advanced Object-Oriented Design & Programming

Introduction

This portfolio reflects on my learning from the Advanced Object-Oriented Design and Programming module. It is structured around four tasks: demonstrating how I achieved the learning outcomes, showcasing key artefacts with critical commentary, reflecting on academic and employability skills, and presenting a Professional Development Plan. I draw on weekly coding exercises and my Capstone Project, **SmartDesk**—an AI-assisted IT service-desk platform that classifies support tickets, estimates service costs and generates operational reports. I chose this domain because its security-sensitive data, interchangeable AI components and evolving reporting requirements provide a realistic context for applying the module's concepts.

Task 1: Summary of Key Learning Outcomes

1.1 Secure coding practices

Early in the module I treated security as something added at the end. The secure-coding units changed that: I now see security as a design property. I applied input validation at trust boundaries, least privilege, and encapsulation that keeps sensitive state private behind validated methods. I also learned that concurrency faults are security faults, because race conditions can corrupt state or expose data.

Practical evidence: in SmartDesk every incoming ticket passes through a validation layer that rejects malformed or oversized payloads before they reach domain objects, database access uses parameterised queries, and API keys are loaded from environment variables rather than hard-coded. In the threading exercise (Artefact 1) I protected a shared counter with a lock after first reproducing the race condition under load.

1.2 Advanced object-oriented principles

I deepened my use of abstraction, encapsulation, inheritance and polymorphism, and learned to treat the SOLID principles as design heuristics rather than rigid rules. The most important shift was favouring composition over inheritance and programming to interfaces, so behaviour can vary without modifying existing classes (Open/Closed Principle).

Practical evidence: SmartDesk's domain layer depends only on abstract base classes (Python's abc module) such as Classifier and PricingStrategy, with concrete implementations injected through constructors. When an early TicketManager class accumulated too many responsibilities, I refactored it into smaller single-responsibility classes for validation, routing and persistence.

1.3 Design patterns for reusable, maintainable and flexible code

Patterns gave me a shared vocabulary for recurring problems, and I learned to select them by the design force they resolve rather than by familiarity.

Advanced design patterns (practical application)

- **Strategy – runtime behavioural flexibility:** triage and pricing algorithms are selected at runtime from client configuration, removing conditional logic scattered across the code.
- **Decorator – separation of concerns:** logging, authorisation and timing wrap the core service without polluting business logic.
- **Visitor – extensibility:** new reports are added as new visitor classes while the ticket and asset hierarchy remains stable.

- **Abstract Factory – scalability:** whole families of AI services (classifier and summariser) can be switched between a cloud provider and a locally hosted model by changing one factory.

AI-oriented OO patterns (conceptual awareness)

I was also introduced to AI-inspired architectural patterns. A **Model Registry** manages the lifecycle of machine-learning models: versioning, metadata, approval stages and rollback. A **Feature Store** centralises feature definitions so a feature such as "tickets raised by this client in 30 days" is computed identically for training and inference, avoiding training–serving skew. I did not build production versions, but I modelled a lightweight ModelRegistry class in SmartDesk that maps model names and versions to classifier objects, which made the link between OO design and MLOps concrete.

1.4 Architectures for large-scale, secure and robust systems

I learned to structure systems in layers (presentation, application, domain, infrastructure) with dependencies pointing inward, so core logic is independent of frameworks and vendors. Robustness came from defensive design: fail-safe defaults, timeouts and retries around external calls, and fallbacks when a dependency is unavailable.

Practical evidence: SmartDesk follows this layered structure. If the AI provider times out, the router falls back to the rule-based triage strategy, so the service degrades gracefully rather than failing. Role-based access is enforced by a decorator at the application boundary.

1.5 Solutions adaptable for AI models and efficient for data science

I learned that AI-readiness is largely an interface problem: if models sit behind stable abstractions they can be retrained, versioned or replaced without destabilising the system. For data efficiency I used generators to stream ticket records and kept data transformation separate from model logic.

Practical evidence: every SmartDesk model implements a common predict() interface and is retrieved from the registry by version, enabling side-by-side comparison of two classifiers. Visitor-based reports export to pandas DataFrames for analysis.

Learning outcome	Main evidence from practical work
Secure coding	Validation layer, parameterised queries, thread-safe ID generator
Advanced OO principles	Interface-driven domain layer; TicketManager refactoring
Design patterns	Strategy, Decorator, Visitor and Abstract Factory artefacts
Large-scale architecture	Layered SmartDesk design, graceful fallback, role decorator
AI-adaptable solutions	Common model interface, ModelRegistry, AI service factories

Task 2: Showcase of Artefacts

The artefacts below come from weekly exercises and the Capstone Project. Code is abridged for length.

Artefact 1: Thread-safe ticket ID generator (coding exercise)

```
import threading

class TicketIdGenerator:
    def __init__(self, start: int = 1000):
        self._next = start
        self._lock = threading.Lock()

    def next_id(self) -> int:
        with self._lock:              # critical section
            current = self._next
            self._next += 1
            return current
```

Without the lock, a stress test running many threads concurrently produced duplicate IDs. Encapsulating both the counter and the lock means callers cannot bypass synchronisation, and a ThreadPoolExecutor-based test now verifies uniqueness.

Artefact 2: Strategy pattern – pricing and triage rules

```
class PricingStrategy(ABC):
    @abstractmethod
    def price(self, ticket) -> float: ...

class StandardPricing(PricingStrategy):
    def price(self, t): return 40.0 * t.hours

class PremiumSLAPricing(PricingStrategy):
    def price(self, t): return 55.0 * t.hours * (1.5 if t.urgent else 1.0)

class Quote:
    def __init__(self, strategy: PricingStrategy): self.strategy = strategy
    def total(self, ticket): return self.strategy.price(ticket)
```

Why chosen: pricing rules and AI triage logic differed by client contract and changed frequently; Strategy replaced a growing if/elif chain. **Benefit:** each rule is unit-tested in isolation and new rules are added without editing Quote. The same structure lets a rule-based triager and an ML triager be swapped at runtime.

Artefact 3: Decorator pattern – logging and security layers

```
class ServiceDecorator(TicketService):
    def __init__(self, inner: TicketService): self._inner = inner

class RequireRole(ServiceDecorator):
    def __init__(self, inner, role): super().__init__(inner); self._role = role
    def submit(self, user, ticket):
        if self._role not in user.roles: raise PermissionError("denied")
        return self._inner.submit(user, ticket)

class AuditLogging(ServiceDecorator):
    def submit(self, user, ticket):
        log.info("submit by %s", user.id)
        return self._inner.submit(user, ticket)

service = AuditLogging(RequireRole(CoreTicketService(), "agent"))
```

Why chosen: logging and authorisation are cross-cutting concerns that should not live inside business logic. **Benefit:** layers are composable and reorderable, and the core service is tested without security or logging noise.

Artefact 4: Visitor pattern – analytics and reporting

```
class ReportVisitor(ABC):
    @abstractmethod
    def visit_ticket(self, t): ...
    @abstractmethod
    def visit_device(self, d): ...

class Ticket:
    def accept(self, v): return v.visit_ticket(self)

class SLABreachReport(ReportVisitor):
    def __init__(self): self.breaches = 0
    def visit_ticket(self, t): self.breaches += t.resolution_hrs > t.sla_hrs
    def visit_device(self, d): pass
```

Why chosen: the ticket/device structure was stable, but reporting requirements changed often. **Benefit:** analytics are separated from domain objects, and a new report is simply a new class. The trade-off is that adding a new element type forces changes to every visitor.

Artefact 5: Abstract Factory – switching AI service providers

```
class AIServiceFactory(ABC):
    @abstractmethod
    def classifier(self) -> Classifier: ...
    @abstractmethod
    def summariser(self) -> Summariser: ...

class CloudAIFactory(AIServiceFactory):
    def classifier(self): return CloudClassifier(os.environ["AI_KEY"])
    def summariser(self): return CloudSummariser()

class LocalAIFactory(AIServiceFactory):
    def classifier(self): return registry.get("triage", version="v3")
    def summariser(self): return LocalSummariser()

factory = LocalAIFactory() if config.data_residency else CloudAIFactory()
```

Why chosen: some clients require data to stay on-premises, and components from different providers must not be mixed. **Benefit:** vendor independence, guaranteed family consistency, and a FakeAIFactory for tests.

2.6 Testing: mocking and AI-driven testing

```
from unittest.mock import Mock

def test_urgent_ticket_routed_to_priority_queue():
    clf = Mock(spec=Classifier)
    clf.predict.return_value = "urgent"
    assert TicketRouter(clf).route(Ticket("Server down")) == "priority"
    clf.predict.assert_called_once()

def test_fallback_when_model_times_out():
    clf = Mock(spec=Classifier); clf.predict.side_effect = TimeoutError
    assert TicketRouter(clf).route(Ticket("Printer jam")) == "standard"
```

Using pytest and unittest.mock, I **isolated classes** from their collaborators; spec= ensures each mock honours the real interface. **AI-dependent components were tested without live models or APIs**: mocks returned deterministic labels and simulated failures via side_effect, making tests fast, free and repeatable.

I also reflected on **AI-assisted testing tools**. An AI coding assistant was useful for **test-case generation**, quickly drafting boundary tests for pricing rules. Combined with property-based testing (Hypothesis), it supported **edge-case discovery**, surfacing inputs such as empty descriptions and zero-hour tickets that I had not considered. Generated tests, once reviewed, were added to the **regression suite** run on every

commit. However, several suggestions merely asserted current behaviour rather than intended behaviour, or duplicated existing tests. I therefore treated AI as a support for OO quality, not a replacement for engineering judgement: I decided what "correct" meant, and the tools helped me check it.

2.7 Capstone Project: SmartDesk

- **Patterns and scalable architecture:** Strategy and Abstract Factory made AI and pricing behaviour pluggable; Decorator kept cross-cutting concerns at the edges; Visitor allowed reporting to grow independently.
- **Testing for robustness:** mock-based unit tests, integration tests per factory, a concurrency stress test and continuous regression testing gave confidence when refactoring.
- **AI-readiness at design level:** models sit behind interfaces, are versioned in a registry and created through factories, with a rule-based fallback strategy, so new models can be adopted without redesign.

2.8 Critical commentary on artefacts

Artefact	OO principles and techniques	Challenges and how I overcame them
Thread-safe generator	Encapsulation, information hiding, context manager	Intermittent race was hard to reproduce; I built a stress test first, then added the lock.
Strategy	Polymorphism, Dependency Inversion, Open/Closed	Deciding who selects the strategy; a small factory driven by contract configuration solved this.
Decorator	Composition, Single Responsibility, Liskov Substitution	Wrapping order changed behaviour; I documented the intended order and tested it.
Visitor	Double dispatch, separation of concerns	Python lacks method overloading and double dispatch was initially confusing; explicit visit_x names and a diagram clarified it.
Abstract Factory	Abstraction, Dependency Inversion, family consistency	Providers had unequal capabilities; I designed to a common interface with optional capability checks.
Mock-based tests	Dependency injection, test doubles	Over-mocking made tests brittle; I limited mocks to architectural boundaries.

2.9 Trade-offs and critical reflection

Pattern selection trade-offs: each pattern resolved one problem while creating another. Visitor eases adding operations but makes adding element types costly; Decorator adds flexibility but deep wrapper chains complicate debugging. In Python, a simple function can often replace a Strategy class, so I reserved full classes for strategies that hold state or configuration.

Testing complexity: abstraction improved testability, but more interfaces meant more test doubles and fixtures to maintain. Concurrency and AI behaviour were the hardest to test deterministically, which I addressed with controlled mocks and repeated stress runs.

Abstraction vs over-engineering: I initially created an Abstract Factory for notification channels with only one implementation. Applying YAGNI and the “rule of three”, I removed it. My working rule became: introduce an abstraction when there is a real second variant or a clear testing need, not in anticipation.

Task 3: Reflection on Skill Development

3.1 Academic skills

Critical thinking and analysis: I learned to break complex problems into forces and responsibilities before writing code, then compare alternative designs against criteria such as coupling, cohesion and testability. Questioning whether a pattern was needed at all, as in the notification factory, became as important as applying one.

Ethical considerations in AI-driven OO design: I became aware that class hierarchies can propagate bias: an assumption encoded in a base class, such as a default priority for a customer category, is silently inherited by every subclass and applied at scale. AI-enabled objects also raise transparency and explainability challenges, so I gave classifiers an explain() method returning the label, confidence and key features, and used a logging decorator to record decisions for auditability. I see the OO designer as responsible for keeping AI components testable, avoiding "black box" coupling by isolating models behind interfaces, and supporting audit through clean, inspectable design.

Problem-solving: I approached coding challenges incrementally: reproduce the problem, write a failing test, implement the smallest fix and refactor. Refactoring the oversized TicketManager under the protection of tests showed me that maintainability improves through small, safe steps.

Research and self-study: I consulted Gamma et al. (1994) for pattern intent, Martin (2017) for architectural layering, OWASP guidance for secure coding and Sculley et al. (2015) on technical debt in machine-learning systems, which shaped my thinking on registries and feature stores. Python documentation and case studies helped bridge theory and practice.

3.2 Employability skills

Time management: balancing coursework with other commitments, I planned weekly goals, timeboxed exercises and split the Capstone Project into milestones (design, core domain, patterns, testing, report) tracked on a Kanban board. Starting the testing work early prevented a last-minute crunch.

Communication and literacy: discussion forums and peer reviews required me to justify design decisions clearly and concisely. UML class and sequence diagrams helped communicate structure to peers, and writing this portfolio strengthened my technical writing.

Resilience and interpersonal skills: setbacks such as the elusive race condition and brittle mocks taught me to persist methodically rather than patch symptoms. In group activities and peer code reviews I learned to give constructive feedback and accept critique of my own designs, which improved both my code and my collaboration. These skills are directly relevant to my aspirations in IT consultancy, where clear design reasoning and client communication matter as much as technical skill.

Task 4: Professional Development Plan (PDP)

4.1 Introduction

This reflection follows Gibbs' Reflective Cycle (Gibbs, 1988): description, feelings, evaluation, analysis, conclusion and action plan. Within the analysis, I draw on Kolb's experiential learning cycle (Kolb, 1984) to explain how my understanding grew through practice rather than reading alone. It covers the academic and employability skills I developed during the module and its Capstone Project (SmartDesk).

4.2 Description and feelings

I began this module with no prior experience of object-oriented programming. Python was also new to me, so I was learning a language and a programming paradigm at the same time. The module moved quickly from core principles to design patterns, concurrency, secure coding and testing. It ended with the SmartDesk Capstone Project, which brought these topics together in one system.

My first feeling was uncertainty. Concepts such as polymorphism and abstraction seemed theoretical when I met them in reading, and Python syntax slowed me down before I could focus on design. I worried that I would understand patterns on paper but not know when to use them.

That feeling changed as the seminars progressed. Seeing a pattern applied to a concrete problem made the theory meaningful. By the end of the module, the main feeling was satisfaction: the separate topics fitted together into a single, working architecture that I could explain.

4.3 Evaluation

The strongest part of the module was the seminars. They were well prepared, and the instructor explained concepts clearly. Most importantly, he showed the anatomy of the final submission early, so I understood what a complete project needed before I started building it. This gave me a map for a large amount of unfamiliar material.

The main difficulty was the double learning curve. Time spent on Python syntax was time not spent on design, and early on I sometimes understood why a pattern existed but could not yet write it confidently. Balancing the module with long working hours made this harder, which I discuss under time management below.

4.4 Analysis: academic skills

Critical thinking and problem-solving

Kolb's cycle reflects the development of my thinking: seminar exercises provided concrete experience, reviewing them supported reflective observation, Gamma et al. (1994) introduced the abstract concepts, and the Capstone enabled active experimentation. The main shift was learning to identify what was likely to change before selecting a design pattern.

In SmartDesk, triage and pricing rules varied across client contracts and were expected to evolve as service requirements changed. I therefore used the Strategy pattern to place these behaviours behind a common interface, allowing individual rules to be replaced or extended without altering the wider system. Reporting requirements also changed frequently, whereas the underlying ticket and device structure remained comparatively stable. This made the Visitor pattern a suitable choice because it allowed new reporting operations to be introduced without adding reporting logic to the domain classes, despite the additional effort required when introducing new element types. Through these decisions, I

learned that design patterns should not be applied simply because they are familiar; each pattern must address a clearly identified source of change and provide benefits that justify its added complexity.

Ethical considerations in AI-driven design

The module showed me that ethics is partly a design decision. If biased weights sit in a base class, every subclass inherits them, and the bias spreads quietly through the hierarchy (Mehrabi et al., 2021). Tight coupling to a model also hides its behaviour, creating "black box" components that are hard to test or audit (Sculley et al., 2015).

I responded through design. Every SmartDesk classifier exposes an explain() method returning the label, confidence and key features, models are versioned in a ModelRegistry and an audit-logging decorator records each decision. Clean interfaces made the system testable with mocks, which is also what makes it auditable.

Research and self-study

Beyond the seminars, I used the module readings, the OWASP guidance on secure coding (OWASP Foundation, 2021) and the Python documentation. The reading that most changed my thinking was Sculley et al. (2015) on hidden technical debt in machine-learning systems. It led me to keep SmartDesk's models behind stable interfaces and a lightweight model registry.

I also used an AI assistant to support the development and organisation of the Capstone code. However, my learning came from examining, running and questioning each component until I could explain it independently. This reinforced a key lesson from the module: AI tools can improve efficiency, but engineering judgement must remain with the developer.

4.5 Analysis: employability skills

Time management

Time management was my main challenge. Long working hours left limited, often tired, study time, while Python and OOP both demanded steady practice. I split the Capstone into milestones tracked on a Kanban board, and starting testing early prevented a last-minute crunch. The lesson I take forward is to plan the hardest learning for the times I am most alert, and to start large tasks early rather than relying on a final push.

Communication and literacy

Writing about the Capstone forced me to explain design choices in plain language, not just implement them. Being able to say why a pattern was chosen, and what it costs, is a skill I can use directly with clients and colleagues. UML class and sequence diagrams helped me explain SmartDesk's structure in discussion forums, and in peer code review I learned to justify trade-offs such as the debugging cost of deep decorator chains.

Resilience and interpersonal skills

Learning two new things at once while working full time required persistence. Early confusion was uncomfortable, but I learned that it was a normal stage of learning rather than a sign of failure. Setbacks such as the race condition in the ticket ID generator taught me to reproduce a problem before fixing it. Clear teaching and the structure of the seminars helped me keep going when progress felt slow.

4.6 Conclusion and action plan

I started the module new to both OOP and Python, and finished able to choose, justify and critique design patterns in a working system. The most important change is in how I think: I now design for change, testability and accountability, not just for code that runs.

Area for improvement	Action	Evidence of progress
Python fluency	Regular coding practice; Python Institute PCAP certification	Certification passed
Pattern judgement	Study Freeman and Robson (2020); refactor one small project per pattern	Refactored projects on GitHub
Testing intelligent systems	Extend mocking and property-based testing to a new AI-dependent project	Test suite with documented edge cases
Time management	Fixed weekly study blocks planned around work shifts	Coursework started at least two weeks before deadlines

Conclusion

This module moved me from knowing patterns and principles to applying them with judgement. SmartDesk demonstrated how secure, layered, pattern-based design and disciplined testing produce systems that are robust today and ready for AI tomorrow, while my reflections on trade-offs and ethics reminded me that good design is ultimately about responsible decisions, not just elegant code.

References

Freeman, E. and Robson, E. (2020) Head First Design Patterns. 2nd edn. Sebastopol, CA: O'Reilly Media.

Freeman, S. and Pryce, N. (2009) Growing Object-Oriented Software, Guided by Tests. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley.

Gibbs, G. (1988) Learning by Doing: A Guide to Teaching and Learning Methods. Oxford: Further Education Unit, Oxford Polytechnic.

Kolb, D.A. (1984) Experiential Learning: Experience as the Source of Learning and Development. Englewood Cliffs, NJ: Prentice Hall.

Martin, R.C. (2017) Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston: Prentice Hall.

Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K. and Galstyan, A. (2021) 'A survey on bias and fairness in machine learning', ACM Computing Surveys, 54(6), pp. 1–35.

OWASP Foundation (2021) OWASP Top 10: 2021. Available at: https://owasp.org/Top10/ (Accessed: 1 October 2026).

Python Software Foundation (2024) unittest.mock — mock object library. Available at: https://docs.python.org/3/library/unittest.mock.html.

Sculley, D. et al. (2015) 'Hidden technical debt in machine learning systems', Advances in Neural Information Processing Systems, 28, pp. 2503–2511.